



پروژه نهایی (فاز اول)

برنامه‌سازی پیشرفته

دانشکده علوم مهندسی

طراح تمرین: سروش نصیری و علی چگینی

بهار ۱۴۰۵



۱. مقدمه و هدف پروژه

Doodle Jump یک بازی کلاسیک «پرش بی‌پایان» است: کاراکتر بازی پیوسته بالا و بالاتر می‌پرد و با هر فرود روی یک سکو، دوباره به هوا پرتاب می‌شود. بازیکن تنها حرکت چپ و راست را کنترل می‌کند – پرش خودکار است. هرچه بالاتر بروید امتیاز بیشتری می‌گیرید، و کافیست یک‌بار از پایین صفحه بیفتید تا بازی تمام شود.

در این پروژه نسخه‌ای ساده‌شده اما کامل این بازی را با زبان `C++` و کتابخانه‌ی گرافیکی `SFML` پیاده‌سازی می‌کنید. هدف این تمرین فقط «کارکردن بازی» نیست؛ بلکه تمرین طراحی شی‌گرا، مدیریت درست حافظه و ساخت یک حلقه‌ی بازی تمیز و قابل‌توسعه است.

این پروژه را می‌توانید به صورت **گروهی** انجام دهید. خروجی فاز اول باید پایه‌ی محکمی باشد که فاز دوم روی آن ساخته می‌شود؛ بنابراین در پایان این فاز، بازی باید یک تجربه‌ی **پرش بی‌پایان** روان را در دنیایی پر از سکوهای متنوع فراهم کند.

⚠ کیفیت طراحی به اندازه‌ی عملکرد اهمیت دارد

راه‌حلی که کار می‌کند اما ساختار ضعیفی دارد، نمره‌ی بسیار کمتری نسبت به یک طراحی تمیز و قابل‌توسعه دریافت می‌کند. تصمیمات طراحی که اکنون راحت به نظر می‌رسند، ممکن است در فازهای بعدی مشکل‌ساز شوند؛ پس از همان ابتدا با دید بلندمدت طراحی کنید.

مرور کلی گیم‌پلی

بازیکن کاراکتری را کنترل می‌کند که به‌طور پیوسته از طریق دنباله‌ای بی‌پایان از سکوها به سمت بالا می‌پرد. ویژگی‌های اصلی گیم‌پلی به این شرح است:

- **پرش خودکار:** بازیکن کلید جهش ندارد؛ هنگامی که کاراکتر روی یک سکو فرود می‌آید، پرش به‌طور خودکار انجام می‌شود.
- **دنیای بی‌پایان:** با بالا رفتن بازیکن، سکوهای جدید به‌طور مداوم تولید می‌شوند.
- **هدف:** تا جای ممکن زنده بمانید و بیشترین امتیاز (ارتفاع) را کسب کنید.
- **پایان بازی:** وقتی بازیکن از ناحیه‌ی قابل‌مشاهده‌ی صفحه پایین‌تر بیفتد، بازی تمام می‌شود.



۲. حلقه‌ی بازی و پنجره

برنامه باید با استفاده از SFML یک پنجره‌ی گرافیکی باز کند که:

- اندازه‌ی ثابت داشته باشد.
 - تا زمانی که کاربر آن را نبندد، به‌طور پیوسته به‌روزرسانی شود.
- حلقه‌ی بازی باید از سه مرحله‌ی مشخص تشکیل شده باشد:

۱. پردازش رویدادها

۲. به‌روزرسانی اشیاء بازی

۳. رندر / ترسیم

۳. کاراکتر بازیکن

بازیکن شیء اصلی قابل‌کنترل است. این کلاس باید حداقل موارد زیر را نگهداری کند:

- موقعیت و سرعت
- امتیاز فعلی

تصاویر کاراکتر

تصاویرِ بلاک‌های کاراکتر در پوشه‌ی `assets/` در اختیار شما قرار داده شده‌اند (به بخشِ تصاویرِ بازی مراجعه کنید). لازم نیست خودتان آن‌ها را طراحی کنید.

کنترل‌های کیبورد

عملکرد	کلید
حرکت به سمت چپ	(Left Arrow) ←
حرکت به سمت راست	(Right Arrow) →

توجه: حرکت مستقل از فریم

حرکت وابسته به فریم قابل قبول نیست. سرعت حرکت باید بر اساس زمان سپری‌شده بین فریم‌ها (delta time) محاسبه شود تا رفتار بازی روی سخت‌افزارها و نرخ‌های فریم متفاوت یکسان بماند.

پیچش صفحه

وقتی بازیکن از یک طرف صفحه خارج شود، بلافاصله از طرف مقابل ظاهر می‌شود.

- این رفتار نباید سرعت بازیکن را تحت تأثیر قرار دهد.



۴. فیزیک و حرکت

۴.۱ سیستم گرانش

بازیکن باید به‌طور مداوم تحت تأثیر گرانش باشد:

- گرانش باید به‌طور مداوم سرعتِ رو به پایینِ بازیکن را افزایش دهد.
- تأثیر گرانش بر سرعت در هر فریم بر اساس `delta time` محاسبه می‌شود.

۴.۲ پرش

بازیکن هنگام فرود روی سکو به‌طور خودکار می‌پرد. پرش فقط در این شرایط فعال می‌شود:

- بازیکن در حال حرکت به سمت پایین باشد.
- بازیکن با سطح بالایی یک سکو برخورد کند.

پرش باید بلافاصله یک سرعتِ رو به بالا به بازیکن اعمال کند.

۴.۳ اسکرول بی‌پایان

یکی از ویژگی‌های اصلی Doodle Jump، توهم حرکتِ بی‌پایانِ رو به بالاست. این باید در پیاده‌سازی شما بازسازی شود:

- بازیکن نزدیکِ وسطِ صفحه ثابت می‌ماند.
- دنیای بازی به سمت پایین اسکرول می‌کند.
- در عمل، اشیاء بازی حرکت می‌کنند، نه بازیکن.

اشیایی که باید تحت تأثیر اسکرول قرار گیرند **سکوها** هستند. اسکرول باید روان و پیوسته به نظر برسد.

۴.۴ سیستم امتیاز

- امتیاز نشان‌دهنده‌ی کل ارتفاعی است که بازیکن طی کرده است.
- امتیاز باید در زمان واقعی روی صفحه نمایش داده شود.
- امتیاز با حرکتِ بازیکن به سمت بالا افزایش می‌یابد.
- طرح بصری نمایش امتیاز به عهده‌ی دانشجو است.



۵. سکوها

سکوها عنصر اساسی گیم‌پلی هستند. بازی باید مجموعه‌ای از سکوهای فعال را مدیریت کرده و با پیشرفت بازیکن، مداوماً سکوهای جدید تولید کند.

قوانین تولید سکو

- هیچ دو سکویی نباید روی هم بیفتند یا یکدیگر را قطع کنند؛ بین هر دو سکو باید حداقل فاصله‌ای ثابت حفظ شود.
- تولید سکو باید به صورت نامحدود ادامه داشته باشد.
- تولید سکو باید تصادفی باشد – طرح ثابت باعث کسر نمره میشود.

⚠ تضمین دسترسی‌پذیری (الزام کلیدی)

بازیکن باید همواره راهی برای ادامه و بالا رفتن داشته باشد؛ بازی هیچ‌گاه و در هیچ ارتفاعی نباید به بن‌بست برسد یا بازیکن را در نقطه‌ای گیر بیندازد. فاصله‌ی عمودی و افقی سکوهای متوالی باید طوری انتخاب شود که با توجه به ارتفاع بیشینه‌ی پرش، رسیدن به سکوی بعدی همیشه ممکن باشد. (دقت کنید که سکوهای شکستنی تضمین کننده‌ی برای ادامه‌ی بازی نیستند)

۵.۱ سلسله‌مراتب کلاس سکو

تمام انواع سکو باید از یک کلاس پایه‌ی مشترک ارث ببرند. کلاس پایه باید رفتار مشترک را شامل شود:

- مدیریت موقعیت.
- رندر (ترسیم گرافیکی).

انواع سکو

نوع سکو	توضیح
عادی (Normal)	سکوی ثابت و پایدار؛ پایه‌ی استاندارد پرش.
متحرک (Moving)	به صورت افقی حرکت می‌کند و در لبه‌های صفحه برمی‌گردد.
شکستنی (Breakable)	پس از یک بار برخورد می‌شکند و دیگر از بازیکن پشتیبانی نمی‌کند.

علاوه بر این، برخی سکوها ممکن است یک فنر (Spring) داشته باشند که به عنوان تقویت‌کننده‌ی پرش عمل می‌کند.



۵.۲ جزئیات سکوی شکستنی

وقتی بازیکن روی سکوی شکستنی فرود می‌آید:

- پرش انجام نمی‌شود.
- سکو شروع به سقوط به پایین می‌کند.
- بازیکن در طول این سقوط به همراه سکو به پایین کشیده می‌شود.
- بازیکن در حین سقوط همچنان می‌تواند با کلیدهای  /  به صورت افقی حرکت کند.
- اگر بازیکن موفق شود به سکوی سالمی در کنار خود برسد و روی آن فرود بیاید، بازی ادامه می‌یابد – این یک فرصت نجات است.
- اگر بازیکن از پایین صفحه خارج شود، بازی پایان می‌یابد.

۵.۳ فنرها

فنرها به عنوان تقویت‌کننده‌ی پرش عمل می‌کنند:

- وقتی بازیکن روی سکوی دارای فنر فرود می‌آید، یک پرش بسیار قوی‌تر رخ می‌دهد.
- فنرها فقط بر روی سکوهای عادی قرار می‌گیرند.
- قرارگیری فنر باید تصادفی باشد – نه روی تمام سکوها.

توجه

رفتار همه‌ی انواع سکو (متحرک، شکستنی و فنردار) باید کاملاً با سیستم برخورد مشترک سازگار باشد؛ یعنی منطق تشخیص فرود نباید برای هر نوع سکو جداگانه بازنویسی شود، بلکه از همان کلاس پایه و سیستم برخورد متمرکز استفاده کند.

برخوردها باید بین موارد زیر مدیریت شوند:

- بازیکن و سکوها.
- بازیکن و فنرها.

حذف سکوهای قدیمی

با بالا رفتن بازیکن و اسکرول رو به پایین دنیا، سکوهای قدیمی به تدریج از لبه‌ی پایین صفحه خارج می‌شوند. این سکوها دیگر دیده نمی‌شوند و کاربردی ندارند؛ پس باید از بازی حذف شوند تا تعداد سکوهای فعال محدود بماند و مصرف حافظه به مرور بی‌رویه افزایش نیابد.



۶. وضعیت‌های بازی

بازی در هر لحظه دقیقاً در یکی از این سه وضعیت قرار دارد و باید بتواند بدون بستن برنامه میان آن‌ها جابه‌جا شود: منوی اصلی، گیم‌پلی و پایان بازی.

۶.۱ منوی اصلی

- عنوان بازی.
- بالاترین امتیاز کسب‌شده تا کنون به صورت برجسته.

دکمه‌های الزامی در منو (با کلیک موس):

دکمه	عملکرد
شروع بازی (Start)	شروع یک جلسه‌ی بازی جدید و انتقال به گیم‌پلی

۶.۲ صفحه‌ی پایان بازی (YOU LOST)

به محض آن‌که بازیکن از پایین صفحه بیفتد، بازی به وضعیت پایان می‌رود. در این لحظه:

- به روزرسانی گیم‌پلی باید متوقف شود.
- یک پیام روشن باخت (مثلاً YOU LOST یا Game Over) در مرکز صفحه نمایش داده شود.
- هم امتیاز نهایی و هم بالاترین امتیاز نمایش داده شود.

دکمه‌های الزامی در صفحه‌ی باخت (با کلیک موس):

دکمه	عملکرد
بازی مجدد (Restart)	شروع مجدد بازی – بدون بازگشت به منو
منوی اصلی (Main Menu)	بازگشت به منوی اصلی

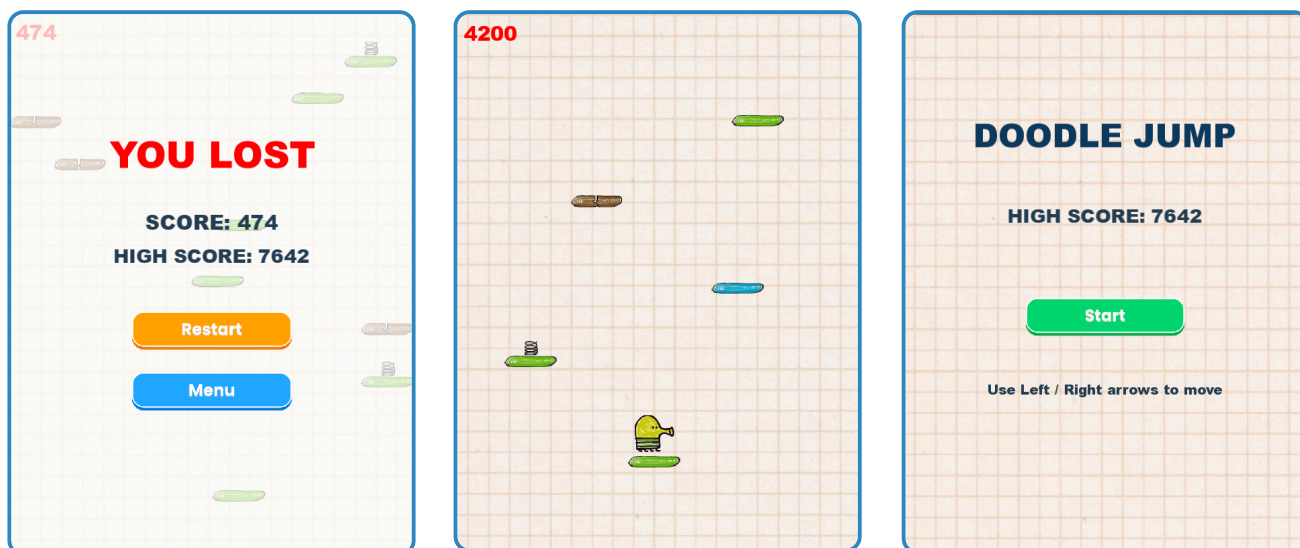
یک چرخه‌ی کامل

سه وضعیت بالا باید با هم یک چرخه‌ی روان بسازند: منو ← گیم‌پلی ← پایان بازی ← (بازی مجدد یا بازگشت به منو). بازیکن باید بتواند بارها و بارها، بدون بستن برنامه، بازی کند.



۶.۳ نمونه‌هایی از اجرای بازی

تصاویر زیر نمونه‌هایی واقعی از اجرای بازی را نشان می‌دهند تا تصویری روشن از ظاهر نهایی مورد انتظار به دست آورید.

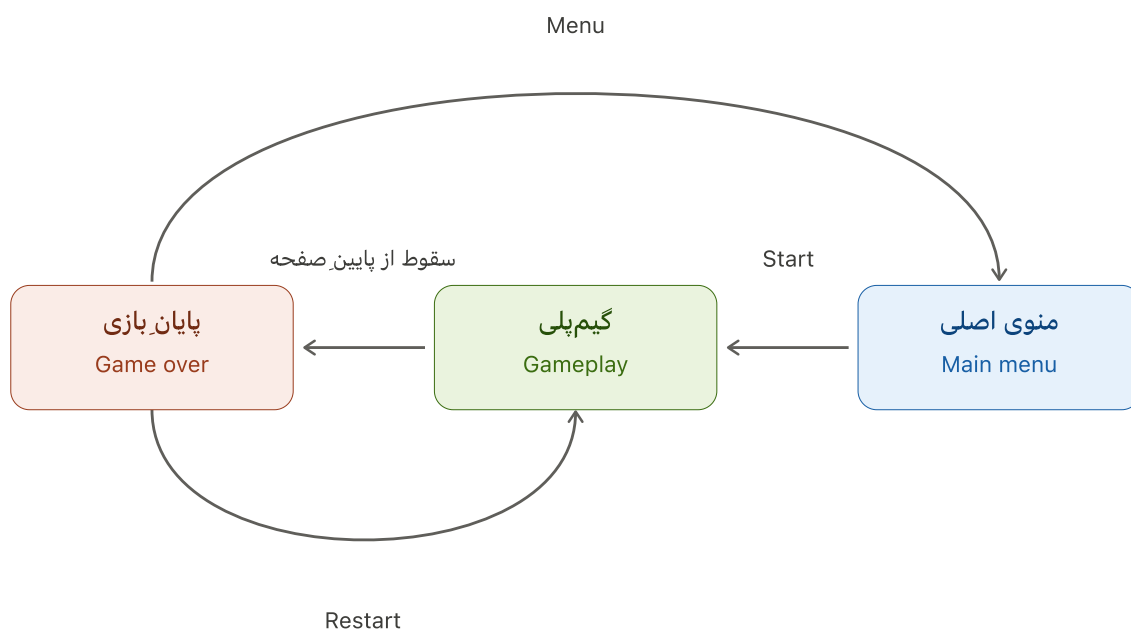


صفحه‌ی باخت: امتیاز، بالاترین امتیاز و گزینه‌ی بازی مجدد.

گیم‌پلی – همه‌ی انواع سکو (عادی، متحرک، شکستی) و فنر در یک قاب.

منوی اصلی همراه با نمایش بالاترین امتیاز.

۶.۴ نمودار وضعیت‌های بازی



چرخه‌ی وضعیت‌ها: منو ← گیم‌پلی ← پایان بازی ← (بازی مجدد یا بازگشت به منو).



۷. الزامات، نکات پیاده‌سازی و تحویل

برای کسب نمره کامل این فاز، رعایت موارد زیر الزامی است:

۱. رعایت اصول شی‌گرایی: تمام منطق برنامه باید در قالب کلاس‌ها پیاده‌سازی شود. متغیرهای کلاس‌ها باید دارای سطح دسترسی مناسب (private یا protected) باشند. نوشتن کدهای طولانی و رویه‌ای در تابع main نمره منفی خواهد داشت.
۲. استفاده از قالب‌ها (Templates): دست‌کم یک کلاس جنریک معنادار با template پیاده‌سازی کنید؛ ترجیحاً قطعه‌ای از زیرساخت که در فازهای بعدی هم به کار بیاید. نمونه‌ی پیشنهادی یک مدیر منابع جنریک (ResourceManager<T>) است که هر نوع دارایی – مانند sf::Texture، sf::Font و در فاز بعد sf::SoundBuffer – را یک‌بار از فایل بارگذاری و کش کند تا از بارگذاری تکراری جلوگیری شود. یک template بی‌استفاده صرفاً برای پر کردن این بند پذیرفته نیست.
۳. تبدیل نوع امن (Type Casting): از static_cast و dynamic_cast استفاده کنید و از کست‌های سبک C پرهیز کنید – مثلاً dynamic_cast برای تشخیص نوع سکو، و static_cast برای تبدیل‌های عددی.
۴. مدیریت حافظه: هیچ‌گونه نشت حافظه‌ای (memory leak) در پایان برنامه نباید وجود داشته باشد؛ هر شیئی که با new ساخته می‌شود باید در جای مناسب با delete آزاد شود.
۵. ساختار چندفایلی (Multi-file): پروژه باید حتماً به صورت چندفایلی (تفکیک .hpp / .cpp) سازماندهی شود.
۶. طراحی برای فازهای آینده: از همان ابتدا با دید بلندمدت طراحی کنید؛ ساختار باید به سادگی قابل گسترش برای ویژگی‌های فازهای بعدی باشد.
۷. رعایت اصول Clean Code الزامی است.

۸. ساختار پوشه‌ی پروژه

ساختار کلی پوشه‌ی پروژه به صورت زیر است: هدرها (.hpp) در پوشه‌ی include/ و پیاده‌سازی‌ها (.cpp) در پوشه‌ی src/ قرار می‌گیرند. سازماندهی داخلی این دو پوشه (زیرپوشه‌ها و تفکیک کلاس‌ها) بر عهده‌ی خود شماست – آن را به گونه‌ای تمیز و قابل توسعه طراحی کنید.

```
simple-doodle-jump/
├── assets/           ← تصاویر بازی (از پیش داده‌شده)
├── fonts/            ← فونت بازی (از پیش داده‌شده)
│   └── ariblk.ttf
├── include/          ← سازماندهی داخلی با شما - هدرهای C++ (.hpp)
├── src/              ← سازماندهی داخلی با شما - پیاده‌سازی‌های C++ (.cpp)
├── highscore.txt     ← نخیرهی بالاترین امتیاز
└── makefile
```